

The program I set out to write is a compiler front end that generates three-address code. It reads C-style expressions and breaks them down into a series of instructions using temporary variables. Assignments store their result directly in the target variable, and expressions store their result in a separate location. Exact precedence, associativity, and parentheses are preserved from the original expression.

From the original cexpr project, the precedence order was left unchanged. This enforced the associativity through explicit grammatical rules. The semantic values are what underwent the most change. Instead of returning a computed value, it now returns an address to the result. Each production generates a new temporary variable, prints the corresponding instructions, frees the operand strings, and returns the new temp. A flag tracks whether the expression was an assignment or not. The dump and clear commands were removed, and some error checking such as overflow and divide-by-zero since there are no mathematical operations being evaluated.

I tested by working through each example in the assignment by hand and comparing the generated instructions line by line against the expected output. When something didn't match, I went back through the grammar rules to figure out which production was being used and in what order, then adjusted accordingly. Additional testing was used from input from the original cexpr assignment.

The main issue I had was implementing the chained assignment portion. I initially had chained assignment print the variable name as the value passed up the chain instead of the temp it was assigned from, which caused other assignments to reference the wrong location. Once I traced through “ $b = c = a - 42$ ” and compared it against the expected output, it was clear the location returned upward needed to stay the temp, not the variable name.